Cadastro de Clientes — Teste Técnico Delphi

Arquitetura em 3 camadas · REST/JSON · FireDAC · PostgreSQL · Threads

Candidato	Marlon Stolf
Entrega	Setembro de 2026
Delphi	Desenvolvido e testado no Delphi 13 (Community). Evita recursos das versões mais recentes (sem *inline var* e sem operador ternário); requer 10.1 Berlin ou superior por usar `THTTPClient.ConnectionTimeout`. Não foi compilado em versões anteriores.
Banco de dados	PostgreSQL 17 (qualquer versão 10 ou superior com a `libpq.dll` 64 bits)
Plataforma	Windows 64-bit
Dependências externas	Nenhuma: apenas RTL, VCL, FireDAC e Indy, que acompanham o Delphi

Sistema de cadastro de clientes (pessoas físicas e jurídicas) e seus endereços, com cliente VCL e servidor REST em Delphi, persistência em PostgreSQL via FireDAC, inclusão em lote de alto desempenho (inclusive por planilha Excel) e atualização dos endereços pelo ViaCEP usando threads.

Resultados medidos

1,8 s
para incluir 50.000 clientes + 50.000 endereços, com validação completa e checagem de CPF/CNPJ único contra ~200 mil clientes

23 consultas
ao ViaCEP para atualizar 55.000 endereços em 2,6 s (4 threads, 0 falhas); cada CEP é consultado uma única vez

0 falhas
nos 4 conjuntos de testes automatizados, incluindo 33 verificações ponta a ponta da API

Requisitos do enunciado e onde são atendidos

Requisito	Atendimento
Banco (tabelas `pessoa`, `endereco`, `endereco_integracao`)	`banco\01_estrutura.sql`, com o DDL idêntico ao enunciado; `banco\00_banco.sql` cria o banco em UTF-8.
Arquitetura em 3 camadas	Apresentação (`cliente\`), negócio (`servidor\Web`, `Controle`, `Regras`, `Integracao`) e dados (`servidor\Dados`, `Infra`), mais contratos comuns em `comum\`.
Comunicação REST com JSON	Servidor Indy com roteador próprio; cliente com `THTTPClient`; contrato JSON único em `UConversorJson`.
Integridade: pessoa sem endereço não existe	Validação compartilhada (cliente e servidor), pessoa e endereços na mesma transação, sincronização de endereços na alteração, sem exclusão isolada de endereço e cascade na exclusão da pessoa. Conferido no banco: 0 pessoas sem endereço.
Persistência com FireDAC	Pool de conexões, `TFDQuery` parametrizado e Array DML.
Insert, Update e Delete	`POST` / `PUT` / `DELETE /pessoas` e as telas de consulta e cadastro de clientes.
Insert em lote performático (50 mil)	Validação prévia (tudo ou nada), IDs reservados em um comando, Array DML em blocos de 5.000 e transação única. Pela tela: clientes de teste gerados ou **importação de planilha Excel**.

Rotina com threads (ViaCEP)	Coordenador + N trabalhadores (`TThread`) com fila `TThreadedQueue<T>`; grava `endereco_integracao` com UPSERT; progresso acompanhado pela tela principal.
Boas práticas	Clean Code, padrões de projeto, POO (herança, encapsulamento, polimorfismo), generics, threads e gerenciamento de memória, detalhados nas seções seguintes.

Manual de avaliação

1. Pré-requisitos

Item	Observação
Delphi 10.1 Berlin ou superior	Testado no Delphi 13 (Community Edition)
PostgreSQL 10 ou superior, **64 bits**	Testado no 17; a `libpq.dll` 64 bits vem junto na instalação
Acesso à internet	Só para a atualização de CEPs (ViaCEP)

2. Criar o banco

No prompt de comando, dentro da pasta do projeto (ajuste a versão do PostgreSQL no caminho):

```
"C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -f banco\00_banco.sql
"C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -d teste_tecnico -f banco\01_estrutura.sql
```

Os dois scripts também podem ser executados pelo pgAdmin ou pelo DBeaver.

3. Configurar o acesso ao banco — `servidor\Servidor.ini`

```
[Servidor]
PortaHttp=9000

[BancoDados]
Servidor=localhost
Porta=5432
Banco=teste_tecnico
Usuario=postgres
Senha=postgres
BibliotecaCliente=C:\Program Files\PostgreSQL\17\bin\libpq.dll
MaximoConexoes=50

[IntegracaoCep]
Url=https://viacep.com.br/ws/%s/json/
Threads=4
TimeoutMs=10000
```

- **Senha**: a senha do usuário `postgres` da sua instalação.
- **BibliotecaCliente**: caminho da `libpq.dll`; troque o `17` se a versão for outra.
- O `cliente\Cliente.ini` (`Url=http://localhost:9000`) só muda se o servidor rodar em outra máquina ou porta. Os dois `.ini` são procurados ao lado do executável e, em seguida, na pasta do projeto.

4. Compilar e executar

1. Abra `TesteTecnico.groupproj` no Delphi. Os projetos já vêm em **Windows 64-bit**.
2. No Project Manager, clique com o botão direito no grupo e escolha **Build All**.
3. Execute o **Servidor**: selecione-o e pressione **Ctrl+Shift+F9** (executa sem depurar). O console mostra `Conectado ao PostgreSQL ...` e `Servidor REST ouvindo em http://localhost:9000`. Se o Windows perguntar sobre o

firewall, permita em rede privada.

4. Execute o **Cliente**: selecione-o e pressione **F9**.

> Abra os projetos pelo `.groupproj` ou pelo `.dproj`, e não pelo `.dpr`. Ao abrir um `.dpr` sem `.dproj`, a IDE gera um projeto só com Win32, e a `libpq.dll` do PostgreSQL atual é 64 bits.

5. Roteiro de uso

O Cliente abre na **tela principal**, com o menu no topo e o rodapé mostrando o banco conectado (versão do PostgreSQL, servidor e nome do banco), a data e o desenvolvedor.

#	O que fazer	O que deve acontecer
1	Menu **Cadastros** → **Clientes**	Abre a consulta de clientes: grade com rolagem contínua e total no rodapé
2	**Inclusão**: **Novo**, natureza *Física*, CPF válido (ex.: `529.982.247-25`), nome, sobrenome e CEP (ex.: `01001-000`); **Salvar**	A tela fecha só depois que o servidor confirma, e o cliente aparece na lista
3	**Validação**: em **Novo**, tente salvar sem endereço, com CPF inválido ou com o CPF/CNPJ de um cliente existente	A tela avisa o problema (no documento repetido, informa qual cliente o usa) e mantém os dados; nada é gravado
4	**Pessoa jurídica**: mude a natureza para *Jurídica*	Os rótulos passam para CNPJ, Razão social e Nome fantasia (ex. de CNPJ: `11.222.333/0001-81`)
5	**Alteração**: selecione um cliente e clique em **Editar** (ou duplo clique). A tela abre com o CEP do primeiro endereço no campo: altere e **Salvar**. **Novo endereço** e **Remover endereço** incluem e excluem	A lista mostra os dados atualizados
6	**Exclusão**: selecione um cliente, **Excluir** e confirme	O cliente sai da lista, com seus endereços
7	**Lote (clientes de teste)**: **Processos** → **Inclusão de clientes em lote**, *Gerar clientes de teste* com **50000**, **Gerar e enviar**	Mensagem com o tempo de gravação no servidor (cerca de 2 s) e o tempo total
8	**Lote (planilha Excel)**: na mesma tela, **Selecionar arquivo** → `modelos\clientes_modelo.xlsx` → **Importar**	Os 200 clientes da planilha são incluídos. Com qualquer problema (CPF inválido, repetido ou já cadastrado, sem CEP), nada é gravado e a tela lista as linhas da planilha a corrigir
9	**Threads/ViaCEP**: **Processos** → **Atualizar endereços pelo CEP (ViaCEP)** e responda **Sim** (somente pendentes)	A barra de progresso avança; ao terminar, a consulta mostra logradouro, bairro e cidade/UF
10	**Reprocessar**: repita o item 9 respondendo **Não** (todos os endereços)	Todos os endereços são reconsultados e atualizados, sem duplicar registros
11	**Busca**: na consulta, digite no campo **Buscar** um nome (`conceição`), uma cidade (`curitiba`), o início de um CPF/CNPJ ou um CEP	A grade mostra só os encontrados, meio segundo após parar de digitar (ou com Enter); **Limpar** volta à lista completa
12	**Rolagem**: role a grade até o fim (barra, roda do mouse ou Ctrl+End)	As linhas são buscadas no servidor em blocos de 500, só quando ficam visíveis

No console do Servidor aparece uma linha por requisição, com método, rota, status e tempo.

6. Testes automatizados (opcional)

Teste	Como rodar	O que cobre

`testes\TestesComum.dproj`	F9 na IDE; não precisa de banco	CPF/CNPJ, pessoa sem endereço, validação de lote, JSON, paginação, leitura da planilha modelo e de uma planilha com erros propositais
`testes\TestesDados.dproj`	F9 na IDE, com o banco criado	Conexão, rollback, CRUD, sincronização de endereços, UPSERT, cascade, lote de 50 mil
`testes\TestarServidor.ps1`	Com o Servidor rodando: `powershell -ExecutionPolicy Bypass -File testes\TestarServidor.ps1`	Endpoints, erros HTTP, integridade, CPF/CNPJ único, busca, lote inválido e lote de 50 mil (33 verificações)
`testes\TestarIntegracaoCep.ps1`	Com o Servidor rodando: `powershell -ExecutionPolicy Bypass -File testes\TestarIntegracaoCep.ps1`	Rotina com threads ponta a ponta, 409, CEP inexistente, dados gravados

7. Problemas comuns

Sintoma	Causa e solução
Servidor: `Cannot load vendor library [libpq.dll]`	Caminho errado em `BibliotecaCliente`, ou projeto compilado em 32 bits (tem que ser Win64)
Servidor: `password authentication failed`	Ajustar `Usuario` / `Senha` no `Servidor.ini`
Servidor: `database "teste_tecnico" does not exist`	Faltou executar o `banco\00_banco.sql`
Cliente: "Não foi possível comunicar com o servidor" ou rodapé com "servidor indisponível"	O Servidor não está rodando ou a porta difere do `Cliente.ini`. Com o Servidor no ar, abrir **Cadastros** → **Clientes** tenta de novo
Porta 9000 ocupada	Trocar `PortaHttp` no `Servidor.ini` e a `Url` no `Cliente.ini`
Atualização de CEPs com falhas	Sem internet, ou o ViaCEP limitou as requisições; basta rodar de novo

Arquitetura

Apresentação — `cliente\`

- Telas VCL: `UPrincipal` (menu e rodapé), `UbrwCliente`, `UcaCliente`, `UcaLote`
- `LibAssincrono`: chamadas em segundo plano; a tela não congela
- `LibPessoa`: acesso REST com `THTTPClient`
- `LibPlanilhaClientes`: leitura de `.xlsx`

Negócio — `servidor\`

- `UServidorHttp` (Indy, uma thread por conexão) → `URoteador`
- `Controle\`: endpoints
- `Regras\` (`Urot`): validações e transações
- `UrotAtualizacaoCep`: coordenador + N `TThread` → `UViaCep`

Dados — `servidor\`

- `UUnidadeTrabalho`: repositórios na mesma conexão e transação
- `Dados\`: repositórios FireDAC com Array DML
- `Infra\`: configuração, pool de conexões, log
- PostgreSQL

Cliente → REST/JSON → Servidor → Unidade de trabalho → PostgreSQL · Rotina de CEP → HTTPS → viacep.com.br

comum — entidades, contrato JSON e validações usados pelos dois lados: o cliente valida antes de enviar e o servidor valida de novo, com a mesma regra.

Nomenclatura

Prefixo	Uso	Exemplos
`Ubrw`	Tela de consulta	`UbrwCliente` (`TfrmBrwCliente`)
`Uca`	Tela de cadastro	`UcaCliente` (`TfrmCaCliente`), `UcaLote` (inclusão em lote)
`Urot`	Regras de negócio	`UrotPessoa`, `UrotAtualizacaoCep`, `UrotSistema`, `UrotValidacaoPessoa`, `UrotValidacaoDocumento`
`Lib`	Apoio às telas	`LibPessoa`, `LibGeradorPessoa`, `LibPlanilhaClientes`, `LibAssincrono`, `LibMensagem`, `LibTexto`
`U`	Demais units	`UPrincipal`, `UConexao`, `URepositorioPessoa`, `URoteador`, `UPessoa`

Nomes em português e curtos. Os fontes não têm comentários: as explicações ficam no README e neste documento. Menus e mensagens da aplicação são todos em português.

Estrutura de pastas

```
TesteTecnico.groupproj   abre todos os projetos na IDE
banco\                   00_banco.sql, 01_estrutura.sql
comum\                   UPessoa, UEntidade, UPagina, UStatusCep, USistema, UConversorJson, UExcecoes,
                         UrotValidacaoPessoa, UrotValidacaoDocumento, LibTexto
servidor\                Servidor.dproj, Servidor.ini, UAplicacao (monta as dependências)
  Web\                   UServidorHttp (Indy), URoteador
  Controle\              UControlePessoa, UControleCep, UControleSistema
  Regras\                UrotPessoa, UrotSistema, UrotAtualizacaoCep (threads)
  Integracao\            UViaCep
  Dados\                 URepositorioBase, URepositorioPessoa, URepositorioEndereco,
                         URepositorioIntegracao, URepositorioSistema, UUnidadeTrabalho
  Infra\                 UConfiguracao, UConexao (pool), ULog
cliente\                 Cliente.dproj, Cliente.ini, UConfiguracaoCliente
  Telas\                 UPrincipal, UbrwCliente, UcaCliente, UcaLote
  Lib\                   LibPessoa, LibGeradorPessoa, LibPlanilhaClientes, LibAssincrono, LibMensagem
modelos\                 clientes_modelo.xlsx (200 clientes, modelo de importação)
testes\                  TestesComum, TestesDados, TestarServidor.ps1, TestarIntegracaoCep.ps1
```

API REST

Todas as respostas são JSON (UTF-8). Os erros seguem o formato `{"erro": "...", "detalhes": ["..."]}`.

Método	Rota	Resposta
`GET`	`/pessoas?pagina=1&tamanho=100&busca=`	`200` página com `dados`, `totalRegistros` e `totalPaginas` (máx. 1000 por página). `busca` opcional: texto em nome, razão social e cidade; número no início do CPF/CNPJ ou CEP
`GET`	`/pessoas/{id}`	`200` pessoa com endereços e dados de integração · `404`
`POST`	`/pessoas`	`201` pessoa criada · `400` validação
`PUT`	`/pessoas/{id}`	`200` pessoa atualizada · `400` · `404`
`DELETE`	`/pessoas/{id}`	`204` · `404`

`POST`	`/pessoas/lote`	`201` `{"inseridos": n, "tempoMs": t}` · `400` com a lista dos registros inválidos
`POST`	`/pessoas/documentos`	`200` recebe uma lista de CPF/CNPJ e devolve os já cadastrados
`POST`	`/enderecos/atualizacao-cep?somentePendentes=true`	`202` rotina iniciada · `409` se já estiver em andamento
`GET`	`/enderecos/atualizacao-cep`	`200` andamento: percentual, CEPs processados, endereços atualizados, não encontrados, falhas
`GET`	`/sistema`	`200` informações do banco para o rodapé: `versao`, `servidor`, `porta`, `banco`

```
{
  "natureza": 1,
  "documento": "529.982.247-25",
  "primeiroNome": "João",
  "segundoNome": "Conceição",
  "enderecos": [ { "cep": "01001-000" } ]
}
```

`natureza`: 1 = física (CPF), 2 = jurídica (CNPJ). As máscaras são removidas pelo servidor. Na resposta, cada endereço traz `integracao` (`uf`, `cidade`, `bairro`, `logradouro`, `complemento`) depois que a rotina do ViaCEP roda.

Decisões técnicas

Integridade: não existe pessoa sem endereço

1. **Validação** (`TPessoaValidador`, usada pelo cliente e pelo servidor): exige ao menos um endereço com CEP de 8 dígitos.
2. **Transação única** (`IUnidadeTrabalho.ExecutarEmTransacao`): pessoa e endereços são gravados juntos; qualquer falha desfaz tudo.
3. **Alteração por sincronização**: exclui os endereços removidos, atualiza os existentes (descartando os dados do ViaCEP quando o CEP muda) e insere os novos, na mesma transação e depois da validação.
4. **Sem exclusão isolada de endereço** na API. A exclusão da pessoa remove os endereços por `ON DELETE CASCADE`.

CPF/CNPJ único

- **Inclusão e alteração**: recusa um documento que já pertence a outro cliente e informa qual; o cliente pode manter o próprio documento na alteração.
- **Lote e planilha**: recusa documentos repetidos no próprio lote e documentos já cadastrados, indicando cada registro (ou linha da planilha).
- Tudo numa única consulta ao banco (`unnest` + junção), eficiente mesmo para 50 mil documentos.
- A estrutura do banco segue exatamente o DDL do enunciado, sem índice único; a garantia é da aplicação. Um índice único seria o reforço natural caso a estrutura pudesse ser alterada.

Inclusão em lote performática

1. Todo o lote é **validado antes** de gravar: se houver registro inválido, nada é gravado e a resposta lista os registros com problema.
2. Os IDs são **reservados em um único comando** na sequence (`nextval(...) from generate_series(1, n)`), sem `RETURNING` linha a linha.
3. Gravação com **FireDAC Array DML** em blocos de 5.000 linhas, com parâmetros tipados antes da execução.

4. **Uma única transação** para pessoas e endereços. O método genérico `ExecutarEmLote<T>` é reaproveitado pelas três tabelas.

Importação por planilha Excel

- Layout de `modelos\clientes_modelo.xlsx`: *Natureza*, *CPF/CNPJ*, *Nome / Razão social*, *Sobrenome / Nome fantasia* e *CEP* (vários separados por `;`). A aba *Instruções* descreve as regras.
- Leitura **sem depender do Excel instalado**: o `.xlsx` é aberto como ZIP (`System.Zip`) e o XML é lido com o OmniXML do Delphi, sem COM, em segundo plano. Os zeros à esquerda de documentos e CEPs digitados como número são recuperados.
- Cada linha passa pela mesma validação do cadastro; os problemas aparecem pelo número da linha da planilha, inclusive os recusados pelo servidor.

Rotina com threads (ViaCEP)

- **Producer-Consumer**: `TCoordenadorAtualizacaoCep` publica o trabalho numa `TThreadedQueue<T>`; os N `TTrabalhadorCep` (4 por padrão) consomem a fila. O encerramento é sinalizado com uma sentinela por trabalhador.
- **Um CEP é consultado uma única vez**: os endereços são agrupados por CEP e o resultado é gravado para todo o grupo num único UPSERT com Array DML (55 mil endereços → 23 consultas).
- **Cada thread tem sua conexão** do pool e seu `THTTPClient`, porque nenhum dos dois é thread-safe.
- **Progresso** protegido por `TCriticalSection`; quem consulta recebe uma cópia e não disputa acesso com as threads.
- **Robustez**: timeout, até 3 tentativas com espera crescente (429/5xx/rede), CEP inexistente contado como "não encontrado", 409 para execução simultânea, cancelamento seguro ao encerrar o servidor e UPSERT para reprocessar sem duplicar.

Consulta de clientes

- **Grade virtual** (`TListView` com `OwnerData`): nasce com o total de linhas e só busca no servidor os blocos de 500 que ficam visíveis, guardando os já carregados.
- **Busca no servidor** (`GET /pessoas?busca=`), com a rolagem sob demanda valendo também no resultado. Com ~150 mil clientes: 30 a 45 ms por número e cerca de 320 ms por texto.
- Parâmetros da URL decodificados em UTF-8 pelo servidor, para buscas com acento.

Padrões de projeto

Padrão	Onde
Repository	`IRepositorioPessoa`, `IRepositorioEndereco`, `IRepositorioIntegracao`, `IRepositorioSistema`
Unit of Work	`IUnidadeTrabalho`: repositórios sobre a mesma conexão e transação
Factory	`IFabricaConexao`, `IFabricaUnidadeTrabalho`, `TFabricaValidadorDocumento`, `TFabricaConsultaCep`
Strategy	`IValidadorDocumento` → `TValidadorCpf` / `TValidadorCnpj`
Template Method	`TValidadorDocumentoModulo11.EhValido`: as classes filhas informam tamanho e pesos
Producer-Consumer	Coordenador + trabalhadores com `TThreadedQueue<T>`
Front Controller	`TRoteador`: rotas, parâmetros e conversão de exceções em respostas HTTP
Mapper / DTO	`TPessoaJson`, `TStatusAtualizacaoCepJson`, `TInformacoesBancoJson`

Composition Root	`TAplicacaoServidor` monta e conecta todas as dependências por construtor

POO, generics e gerenciamento de memória

- **Herança**: `TEntidade` → `TPessoa` / `TEndereco`; `TRepositorioBase` → repositórios; `TValidadorDocumentoModulo11` → CPF/CNPJ; `TThread` → coordenador e trabalhadores.
- **Polimorfismo**: métodos `virtual; abstract` e dependências sempre por interface (`IRegraPessoa`, `IConsultaCep` etc.).
- **Encapsulamento**: campos `private` expostos por `property`; regra única de posse dos objetos.
- **Generics**: `TPagina<T>`, `ExecutarEmLote<T>`, `TAssincrono.Executar<T>`, `TThreadedQueue<T>`, `TObjectList<T>`, `TObjectDictionary<K,V>`.
- **Criação e destruição de objetos**: `try..finally` em toda criação; `try..except Free; raise` em funções que devolvem objeto; listas donas dos itens; interfaces com contagem de referência; `ReportMemoryLeaksOnShutdown` em Debug.

Outras decisões

- **Indy + roteador próprio** em vez de DataSnap ou frameworks de terceiros: funciona em qualquer edição do Delphi, sem bibliotecas externas.
- **Pool de conexões FireDAC**: o Indy atende cada conexão numa thread, e cada uma pega sua conexão do pool.
- **Arquivos-fonte em UTF-8 com BOM**, para que os acentos das mensagens sejam lidos corretamente pelo compilador.